Each `ActorSystem` has only one instance of this type, and this actor has a double task. It is the root of the hierarchy of all actors and the mechanism that controls the entire system. This chapter is about this hierarchy, why it is important, and how to implement it.

To learn this, you will see an application that parses text. It is modeled with a manager that delegates the parsing of each text to an actor: a parser. The manager specializes in creating parsers and delegating tasks to them, while the parsers do only one thing and do it well: process text. Let's put this into practice by creating a hierarchy of actors that together serve this one purpose.

NOTE The source code for this chapter is available at www.manning.com/books/akka-in-action-second-edition or https://github.com/franciscolopez sancho/akka-topics/tree/main/chapter03. You can find the contents of any snippet or listing in the .scala file with the same name as the class, object, or trait. For example, you can find the snippet with `object ErrorKernelApp` in the ErrorKernelApp.scala file.

3.1 Simple parser example

The actors in this application are a guardian, which creates a manager, which in turn creates several workers. The guardian passes the list of texts to be parsed to the manager, which passes them to the workers to parse. The hierarchical structure is shown in figure 3.1.